What Claude needs only sometimes

What does not go in CLAUDE.md, and how to make sure Claude reads it

Florian Gärtner

Where we are today

	What Day 1 taught
	What Claude needs in every session
→	**What Claude needs only sometimes** ← you are here
	Your own skill
	Lunch
	A history you can go back to
	Where does this file live?
	Commit first, then let Claude move things. Then GitHub
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Choose** where a piece of knowledge belongs: read every session, read when relevant, or read when pointed at.
2. **Have Claude add** a pointer to `CLAUDE.md` that says where a note is and when to read it.
3. **Explain** why a note read on demand can be missed, and what makes it likelier to be read.

Beyond `CLAUDE.md`

What did not go in

Your `CLAUDE.md` files, your global one and your projects', hold what Claude needs in every session. These do not fit there:

- what Claude picks up about how you work, which nobody writes down
- a four-page procedure for cleaning a survey export, needed only when you clean one
- where you stopped last night

Four notebooks

Notebook	Who writes it	When it is read
`CLAUDE.md`	Claude drafts, you approve	always
Auto memory	Claude, unreviewed	index always, notes when needed
Skills and your `/commands`	Claude drafts, you approve	description always, the rest when used
Notes and the handover	Claude, when you ask	when pointed at

Auto memory

Claude’s own notes

Claude keeps notes for itself as it works: your preferences, corrections you gave it, decisions it cannot read off the files.

You say	It goes to
“Remember that `income` is in thousands.”	auto memory
“Add this to CLAUDE.md.”	your `CLAUDE.md`

Only the **index** is read at every start (its first 200 lines). Each note behind it is read when Claude thinks it needs it.

Read what Claude wrote

Type `/memory` to open your `CLAUDE.md` files or Claude's notes folder, or to switch auto memory off.

The notes are plain text. Ask Claude to fix or delete what is wrong.

The notes also grow on their own, and only the index's first 200 lines are read at the start. A note listed further down is still there, but Claude no longer sees it at the start. When the folder gets long, ask Claude to merge what overlaps and drop what is out of date.

> **Rule of thumb**
>
> When Claude says it saved a memory, ask to see it. A wrong note carries the same weight as a line you approved.

What has Claude noted about your project?

Your project · free time

- Type `/memory` and open Claude’s notes folder, or ask Claude what it has noted.
- Is anything in the notes wrong? Ask Claude to fix it.

Skills and your `/commands`

`CLAUDE.md` is getting long

Suppose you have a four-page procedure for cleaning a survey export. Claude needs it only when you clean one.

In `CLAUDE.md`, all four pages go in on every turn of every session.

What would you want instead?

A skill: a manual Claude opens when needed

A **skill** is a folder with a `SKILL.md` file: a short description, then the instructions.

```
---
description: Cleans a raw survey export. Use when the user
  asks to clean, recode or import a survey file.
---
1. Drop the test responses (status = "preview").
2. Recode the Likert items to 1-5.
...
```

At the start of a session Claude sees **only the description**. Claude opens the rest when a task matches.

Your own commands are skills

The folder's name becomes a command. `.claude/skills/clean-survey/` gives you **`/clean-survey`**.

Type it, and Claude follows that procedure now, whether or not it would have thought of it.

A procedure you explain again and again belongs in a skill.

> ⚠ **Use only skills you have read**
>
> A skill someone else wrote gives Claude instructions, and it can let Claude use tools without asking you.

Your kit’s skills and commands

Skill	For
`empirical-analysis`	estimation, identification, plots
`analysis-plan`	pre-registrations, power
`research-paper`	LaTeX tables and figures
`author`	prose others will read
`pre-submission-audit`	the last check before submitting
`project-organizer`	script pipelines, folder layout
`coder`	writing and debugging R and Python
`auditor`	an independent review
`skill-builder`	writing a skill of your own

Command	Use it to
`/startup`	see where the project stands
`/session-end`	write the handover, save the work
`/newproject`	set up a new project
`/what-slipped`	have a fresh reviewer check the work
`/explain`	have the last thing explained plainly
`/panic`	save everything before the session dies

Claude opens a skill when the task matches. You can also call one by name: `/auditor`.

A work in progress: let your Claude suggest changes, and change them to fit your work.

Notes and the handover

Now, then, and why

Note	Holds	Claude …
`next_steps.md`, `OPEN_ISSUES.md`, the long-term list	what is true now	rewrites it, and deletes what is done
a log, `session_log.md`	what happened, and when	only ever adds to it
a discussion file per decision, `decision_<topic>.md`	what was decided, and why	writes it when you decide

All of them live in `claude/`. The first kind is a snapshot, not a log: history goes to the log, which nothing reads unless you ask.

> ⚠ A file that is read at every start and only grows stops being read.

A note for next time

At the end of a session, ask Claude:

> *"Write the state of this work to `claude/next_steps.md`: what is done, and what is next."*

With your kit, `/session-end` does this for you, and saves the work.

Check it the next time, in a new session:

> *"Read `claude/next_steps.md`. Where did we stop, and what would you do first?"*

If you could not pick up the work from that answer, tell Claude what was missing.

Tomorrow, the new session can read where you stopped, if it knows the file is there.

What happens to an open issue

`claude/OPEN_ISSUES.md` holds what is open now. Ask Claude to note an issue the moment you notice it.

- **Resolved?** It comes out.
- **Can wait?** It moves to a second list, `claude/long_term_open_issues.md`, read only when you ask.
- **Too big for a few lines?** It gets its own file, and the list keeps one line pointing at it.
- **Can never be finished?** Then it is not an issue. A standing rule goes in a file Claude always reads: `CLAUDE.md` or a rules file. A recurring task's steps go in a skill. You start it when it is due, or `/loop` runs it at an interval while the session stays open.

A log: what happened, and when

At the end of a session, ask Claude:

> *"Add today's entry to `claude/session_log.md`: the date, what we did, and which decision files we wrote. Change nothing above it."*

With your kit, `/session-end` adds the entry for you.

Weeks later, the log answers what no snapshot can:

> *"Read `claude/session_log.md`. When did we drop the 2020 wave, and why?"*

It keeps even what changed no file: what you read, and what you tried and dropped.

A discussion file per decision

When you settle a question that took weighing (a sample restriction, an estimator), ask Claude:

> *"Write `claude/decision_sample_restriction.md`: the question, the options, what we chose and why, and what we rejected."*

With your kit, Claude usually offers to write one when such a question is settled, and `/session-end` writes any that are still missing.

A year later, the reason is what you need, and nobody remembers it.

The file records the decision. What follows from it goes in `OPEN_ISSUES.md`.

Making sure it gets read

On demand means Claude decides

`CLAUDE.md`, the auto-memory index and the skill descriptions are read in every session. Everything else is read only when Claude decides it is needed. Usually Claude decides right. Sometimes not:

- You ask for a table for the paper. Your skill for paper tables is there, and Claude formats the table without opening the skill.
- You ask to continue where you stopped. Claude starts from the files it sees and never opens yesterday's note.

On demand takes no room until it is read, and Claude can miss it.

Fix 1: a pointer

A pointer is a line in `CLAUDE.md` that says where a note is and when to read it.

```
- What is open: `claude/OPEN_ISSUES.md`. Read it when I ask what is open.
- What happened: `claude/session_log.md`. Read it when I ask what we did.
- Why we decided: `claude/decision_<topic>.md`. Read it before changing that topic.
- The codebook: `docs/codebook.md`. Read it before recoding a variable.
```

The pointer is read in every session, the note only when it is needed. A skill works the same way.

> **Rule of thumb**
>
> When Claude writes a note, ask for the pointer to it too.

Fix 2: you call it

Some moments only you know about: the end of a session, a draft about to go to a co-author. For those, do not leave the decision to Claude. Call the skill yourself:

```
/session-end
```

YOUR KIT

Claude then reads that skill and the notes it names, here the handover note and the open issues, and brings them up to date. Every time, whether or not Claude would have thought of it.

The catch: you have to remember to type it.

Fix 3: a hook

A **hook** is a small script that Claude Code runs at a fixed moment: when a session starts, when you send a prompt, before a command runs.

Claude does not decide whether it runs. It runs every time that moment comes.

Your kit has two. When a session starts, `scripts/session-start.ps1` pastes in the project's `claude/next_steps.md`. With each prompt you send, `scripts/context-in-prompt.ps1` adds how full the session is.

A hook can stop Claude, or remind it

	What the hook does	What Claude does
Stop	refuses a command before it runs: *no edits in* `data/raw/`	cannot run it, reads the reason, may try another way
Remind	adds a line at the moment it applies: *"You are editing R code: load the R skill."*	follows it, or sets it aside when it does not fit

A reminder lands right next to the step it is about. `CLAUDE.md` sits at the start of the input, before the whole conversation.

Claude Code docs, "Extend Claude Code": "An instruction like 'never edit .env' in CLAUDE.md or a skill is a request, not a guarantee. A PreToolUse hook that blocks the edit is enforcement."

A hook sees the moment, not the meaning

A hook sees what the moment hands it: which tool Claude uses, the command, the file.

- **A stop catches only what it names.** A hook on Claude's edits does not see an R script, run by Claude, that overwrites the same file.
- **A reminder sometimes fires on the wrong file.** Claude can set it aside. A stop that fires wrongly blocks good work.
- **Every hook is a script.** Someone keeps it working on Windows and on Mac, and it runs with your rights. Use only hooks you have read.

> ⓘ **Only a stop is not a request**
>
> `CLAUDE.md`, skills, notes and reminders ask Claude to do something. A hook that stops a command holds whether Claude agrees or not.

What to take away

1. Claude does not remember. **Memory is a file that gets read next time.**
2. When Claude says it saved a memory, ask to see it. A wrong note carries the same weight as a line you approved.
3. A skill's instructions and your notes take no room until they are read, and are read only if something leads Claude there.
4. When Claude writes a note, ask for the pointer to it too.
5. A hook that stops a command holds. A hook that reminds is still a request.